Transpose Matrix

AlgoExpert · Easy · Arrays

The problem

- Flip a 2D matrix over its main diagonal.

Intuition

- Transposing swaps rows and columns, so the cell at row i, column j moves to row j, column i.

Brute force vs optimal

- Only one sensible approach: copy every cell once
- Must allocate a new C×R matrix (shape changes)
- In-place only works for square matrices

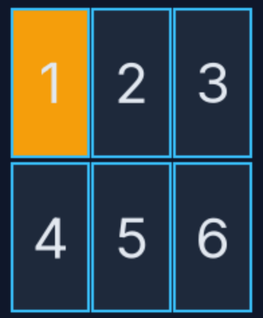
Approach

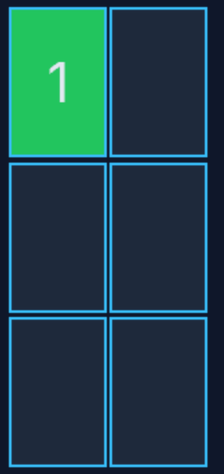
- 1. Allocate a matrix with the dimensions swapped
- 2. Copy each cell to its mirrored position
- 3. Return the new matrix

Worked example

input 2×3

output 3×2

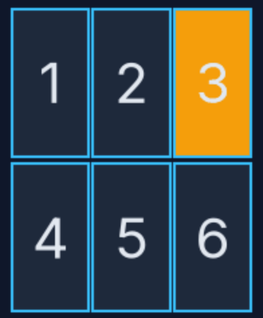




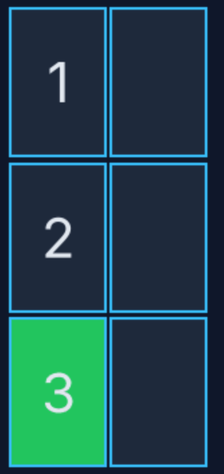
m[0][0] = 1 → t[0][0]

Worked example

input 2×3



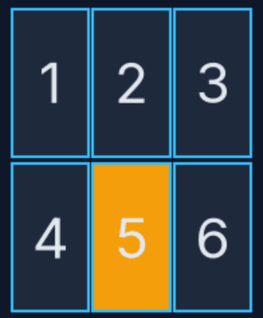
output 3×2



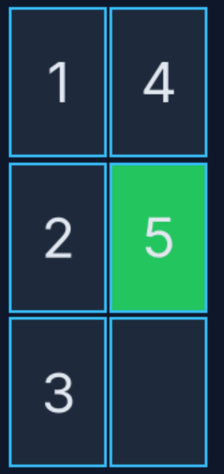
m[0][2] = 3 → t[2][0]

Worked example

input 2×3



output 3×2



m[1][1] = 5 → t[1][1]

Worked example

input 2×3

1	2	3
4	5	6

output 3×2

1	4
2	5
3	6

m[1][2] = 6 → t[2][1]

Java solution

```java
public class TransposeMatrix {
    public static int[][] transposeMatrix(int[][] matrix) {
        int rows = matrix.length, cols = matrix[0].length;
        int[][] out = new int[cols][rows];
        for (int r = 0; r < rows; r++) for (int c = 0; c < cols; c++) out[c][r] = matrix[r][c];
        return out;
    }
}
```

Complexity

- Time: O(w*h)
- Space: O(w*h)

Edge cases & pitfalls

- Swap the dimensions when allocating
- 1×n and n×1 inputs
- Don't mix up rows and columns in the loop

Interview tips & follow-ups

- Follow-up: in-place transpose of a square matrix (swap i<j)
- Follow-up: rotate 90° = transpose + reverse rows

Recap

- Transposing swaps rows and columns, so the cell at row i, column j moves to row j, column i.
- Complexity: O(w*h)